RADWARE – GROOMING DOCUMENT

Role: Internship Engineer – Software (Java Backend)

Written Test + Technical Interview Preparation Guide

Prepared and improved based on the provided Radware Java Backend content.

Company Overview

Radware is a global cybersecurity and application delivery company that helps organizations keep their applications, APIs, and networks secure and available, even during heavy traffic or cyberattacks.

It builds backend and cloud systems that must be reliable, scalable, secure, and performance-focused.

In Simple Terms

Radware works on systems where backend development, REST APIs, databases, networking, performance, and security come together.

For an **Internship Engineer – Software (Java Backend)** role, this means you should be strong in Java fundamentals and also understand how backend systems work in real environments.

You are not preparing only for Java syntax.

You are preparing to explain how Java backend applications are built, how APIs work, how data is stored, how errors are handled, and how systems are debugged.

1. Role Overview

The role of **Internship Engineer – Software (Java Backend)** is for students who want to learn backend development using Java, Spring Boot, REST APIs, SQL, and basic system concepts.

This role is not only about writing simple Java programs.

It is about understanding how backend services are designed and how they behave when users, APIs, databases, and networks interact.

From the JD and role expectations, you should be comfortable with:

Core Java

OOP concepts

Strings

Collections

Exception handling

Basic multithreading

Spring Boot basics

REST APIs

JSON

SQL and relational databases

JPA/Hibernate basics

Git

Debugging

Basic OS and networking

Since this is an internship role, the company will not expect you to design complex production architecture.

But they will expect clear fundamentals.

The most important point is this:

You do not need to sound like a senior backend architect.

You need to sound like a fresher with clear basics, clean thinking, and a strong learning mindset.

2. Work Nature

As a Java backend intern, your work may involve supporting the engineering team in building and maintaining backend services.

You may work on:

Creating or modifying REST APIs

Writing business logic in service classes

Working with database tables

Writing SQL queries

Using Spring Boot annotations

Handling JSON requests and responses

Debugging API failures

Reading logs

Writing small bug fixes

Understanding existing backend code

Testing API behavior

Working with Git

Collaborating with frontend, QA, and DevOps teams

Initially, you may not own critical production modules alone.

That is normal.

Your main work is to observe, understand, assist, practice, and gradually take ownership of small backend tasks.

A good backend intern continuously asks:

What is this API supposed to do?

Which layer is failing?

What do the logs say?

Is the issue in code, database, configuration, or network?

What changed recently?

Can this be handled better next time?

This mindset matters a lot in backend interviews.

3. Selection Process

The selection process will most likely have two major stages.

Round 1: Written / Online Test

The written test will check whether your fundamentals are clear.

Expected areas:

Core Java

OOP

Strings

Collections

Exception handling

Java output prediction

Spring Boot basics

REST API basics

SQL

OS basics

Networking basics

Git

Simple coding logic

Question formats may include:

MCQs

Output prediction questions

Short concept questions

Small Java code snippets

SQL queries

Scenario-based questions

The written test is not designed to test deep microservices architecture.

It is designed to check whether you understand basic backend concepts and can reason about code.

Round 2: Technical Interview

The technical interview will check how clearly you can explain concepts and how logically you can debug problems.

Expected areas:

Core Java explanation

OOP with examples

Collections usage

Exception handling

String handling

Basic multithreading

Spring Boot and REST APIs

Controller-service-repository layers

SQL and database design

HTTP methods and status codes

OS and networking basics

Git usage

Project discussion

Debugging scenarios

The interviewer will check:

Can you explain concepts simply?

Can you connect concepts with backend usage?

Can you debug step by step?

Can you explain your project clearly?

Are you honest when you do not know something?

Are you willing to learn?

For this internship, your clarity and thinking process matter more than fancy words.

4. Preparation for Written Test

The written test should be prepared topic-wise.

Do not randomly revise Java.

Prepare in this order:

Core Java

OOP

Strings

Collections

Exceptions

Spring Boot

REST APIs

SQL

OS and networking

Git

Output-based questions

Simple coding logic

Focus on why a concept is used, not only the definition.

5. Written Test Preparation Questions

5.1 Core Java and OOP – Most Expected MCQs

Question 1. Which feature of OOP helps hide internal details and expose only necessary functionality?

A. Inheritance

B. Polymorphism

C. Abstraction

D. Encapsulation

Answer: C

Explanation:

Abstraction hides implementation details and shows only essential functionality. In Java, abstraction can be achieved using abstract classes and interfaces.

Interview Relevance:

This question checks whether you understand the meaning of abstraction beyond just memorizing the OOP pillars.

Preparation Tip:

Remember: abstraction focuses on “what it does,” not “how it does it.”

Question 2. Which OOP concept represents the same method name behaving differently?

A. Encapsulation

B. Polymorphism

C. Abstraction

D. Compilation

Answer: B

Explanation:

Polymorphism means one name, many forms. In Java, it can happen through method overloading and method overriding.

Example:

draw() may behave differently for Circle, Rectangle, and Triangle objects.

Question 3. Which keyword refers to the current object in Java?

A. this

B. super

C. self

D. object

Answer: A

Explanation:

this refers to the current object. It is commonly used to access instance variables, call current class methods, and resolve naming conflicts between local and instance variables.

Question 4. Which keyword refers to the immediate parent class object?

A. this

B. super

C. parent

D. base

Answer: B

Explanation:

super is used to refer to the immediate parent class. It can be used to access parent class variables, methods, and constructors.

Question 5. Which statement about constructors is true?

A. Constructors must always be private

B. Constructors have no return type

C. Constructors must be static

D. Constructors must return an object

Answer: B

Explanation:

Constructors do not have a return type. They are called automatically when an object is created.

Question 6. Can a constructor be overloaded?

A. Yes

B. No

Answer: A

Explanation:

Constructor overloading means creating multiple constructors with different parameter lists.

Example:

```
class Student {
   Student() {
   }

   Student(String name) {
   }
}
```

Question 7. Which concept is used to reuse parent class properties in child class?

A. Encapsulation

B. Inheritance

C. Abstraction

D. Exception handling

Answer: B

Explanation:

Inheritance allows one class to acquire properties and methods of another class.

Question 8. Which of the following supports multiple inheritance in Java?

A. Class

B. Interface

C. Constructor

D. Package

Answer: B

Explanation:

Java does not support multiple inheritance through classes to avoid ambiguity, but it supports multiple inheritance through interfaces.

Question 9. What is encapsulation?

A. Binding data and methods together

B. Hiding classes from compiler

C. Writing only static methods

D. Running multiple threads

Answer: A

Explanation:

Encapsulation means binding data and methods together in a class and controlling access using access modifiers.

Question 10. Which access modifier provides the highest restriction?

A. public

B. protected

C. private

D. default

Answer: C

Explanation:

private members are accessible only within the same class.

5.2 Java Strings – Most Expected MCQs

Question 11. Are String objects immutable in Java?

A. Yes

B. No

Answer: A

Explanation:

Strings are immutable in Java. Once created, their value cannot be changed. Any modification creates a new String object.

Question 12. What is the output?

```
String s1 = "Java";
String s2 = "Java";
System.out.println(s1 == s2);
```

A. true

B. false

Answer: A

Explanation:

Both string literals refer to the same object in the String pool.

Question 13. What is the output?

```
String s1 = new String("Java");
String s2 = "Java";
```

System.out.println(s1 == s2);

A. true

B. false

Answer: B

Explanation:

new String("Java") creates a new object in heap memory, while "Java" refers to the String pool. So references are different.

Question 14. Which method should be used to compare String values?

A. ==

B. equals()

C. compareRef()

D. check()

Answer: B

Explanation:

equals() compares actual content. == compares references.

Question 15. What is the return type of length() method in String?

A. long

B. int

C. double

D. short

Answer: B

Explanation:

length() returns an integer value representing the number of characters in the String.

5.3 Collections – Most Expected MCQs

Question 16. Which collection allows duplicates and maintains insertion order?

A. HashSet

B. TreeSet

C. HashMap

D. ArrayList

Answer: D

Explanation:

ArrayList allows duplicate elements and maintains insertion order.

Question 17. Which collection stores unique elements with no guaranteed order?

A. ArrayList

B. HashSet

C. LinkedList

D. Vector

Answer: B

Explanation:

HashSet does not allow duplicates and does not guarantee insertion order.

Question 18. Which collection stores key-value pairs?

A. ArrayList

B. HashSet

C. HashMap

D. Stack

Answer: C

Explanation:

HashMap stores data in key-value format.

Question 19. Which collection is synchronized?

A. ArrayList

B. HashMap

C. Vector

D. HashSet

Answer: C

Explanation:

Vector is synchronized, while ArrayList is not synchronized.

Question 20. Which is better for frequent random access?

A. ArrayList

B. LinkedList

Answer: A

Explanation:

ArrayList is backed by an array, so index-based access is faster.

Question 21. Which is better for frequent insertion and deletion in the middle?

A. ArrayList

B. LinkedList

Answer: B

Explanation:

LinkedList is better for frequent insertions and deletions because elements are linked using nodes.

Question 22. Does HashMap allow null keys?

A. Yes, one null key

B. No null key

C. Multiple null keys

D. Only null values

Answer: A

Explanation:

HashMap allows one null key and multiple null values.

Question 23. Does HashSet allow duplicate values?

A. Yes

B. No

Answer: B

Explanation:

HashSet stores only unique elements.

5.4 Exception Handling – Most Expected MCQs

Question 24. Which of the following is a checked exception?

A. NullPointerException

B. ArithmeticException

C. IOException

D. ArrayIndexOutOfBoundsException

Answer: C

Explanation:

IOException is a checked exception. Checked exceptions are checked at compile time.

Question 25. Which of the following is an unchecked exception?

A. IOException

B. SQLException

C. FileNotFoundException

D. NullPointerException

Answer: D

Explanation:

NullPointerException is an unchecked exception because it occurs at runtime.

Question 26. What is the main purpose of the finally block?

A. To catch exceptions

B. To execute cleanup code

C. To create objects

D. To stop program execution

Answer: B

Explanation:

The finally block executes whether an exception occurs or not. It is often used for cleanup operations like closing resources.

Question 27. What is the difference between throw and throws?

A. throw declares exception, throws creates exception

B. throw is used to explicitly throw an exception, throws declares possible exceptions

C. Both are same

D. Both are used only for checked exceptions

Answer: B

Explanation:

throw is used inside a method to throw an exception. throws is used in method signature to declare exceptions.

Question 28. What is the problem with catching generic Exception everywhere?

A. It improves performance

B. It hides real issues and makes debugging harder

C. It is mandatory

D. It always prevents bugs

Answer: B

Explanation:

Catching generic Exception everywhere can hide the actual type of error and make debugging difficult.

5.5 Spring Boot and REST – Most Expected MCQs

Question 29. Why is Spring Boot preferred over plain Spring?

A. It removes Java

B. It reduces boilerplate configuration

C. It runs only desktop apps

D. It avoids REST APIs

Answer: B

Explanation:

Spring Boot provides auto-configuration, embedded server support, and faster setup for backend applications.

Question 30. Which annotation is used to create REST APIs returning JSON?

A. @Controller

B. @RestController

C. @Service

D. @Repository

Answer: B

Explanation:

@RestController combines @Controller and @ResponseBody, making it suitable for REST APIs.

Question 31. Which annotation maps HTTP GET requests?

A. @PostMapping

B. @GetMapping

C. @PutMapping

D. @DeleteMapping

Answer: B

Explanation:

@GetMapping is used to handle GET requests.

Question 32. Which annotation is used to read JSON request body?

A. @PathVariable

B. @RequestParam

C. @RequestBody

D. @Autowired

Answer: C

Explanation:

@RequestBody binds the JSON request body to a Java object.

Question 33. Which annotation binds URL path values to method parameters?

A. @RequestBody

B. @PathVariable

C. @RequestHeader

D. @Service

Answer: B

Explanation:

@PathVariable is used to extract values from the URL path.

Question 34. Which annotation marks a class as service layer component?

A. @Service

B. @Entity

C. @Table

D. @Column

Answer: A

Explanation:

@Service indicates that the class contains business logic.

Question 35. Which annotation marks a JPA entity?

A. @Entity

B. @RestController

C. @Service

D. @Bean

Answer: A

Explanation:

@Entity maps a Java class to a database table.

Question 36. Which HTTP method is used to fetch data?

A. GET

B. POST

C. PUT

D. DELETE

Answer: A

Explanation:

GET is used to retrieve data from the server.

Question 37. Which HTTP method is used to create a new resource?

A. GET

B. POST

C. DELETE

D. TRACE

Answer: B

Explanation:

POST is commonly used to create new resources.

Question 38. Which HTTP method is commonly used to update an existing resource completely?

A. GET

B. POST

C. PUT

D. DELETE

Answer: C

Explanation:

PUT is generally used for full updates of an existing resource.

Question 39. Which status code means successful request?

A. 200

B. 404

C. 500

D. 403

Answer: A

Explanation:

200 means OK or successful request.

Question 40. Which status code means resource not found?

A. 200

B. 201

C. 400

D. 404

Answer: D

Explanation:

404 means the requested resource was not found.

Question 41. Which status code means internal server error?

A. 200

B. 201

C. 404

D. 500

Answer: D

Explanation:

500 means the server encountered an unexpected error.

5.6 SQL – Most Expected MCQs

Assume tables:

Users(id, name, city)

Orders(id, user_id, amount)

Question 42. Which query gives total amount spent by each user?

A. SELECT user_id, SUM(amount) FROM Orders GROUP BY user_id;

B. SELECT user_id, amount FROM Orders;

C. SELECT SUM(amount) FROM Orders;

D. SELECT * FROM Orders;

Answer: A

Explanation:

GROUP BY user_id groups orders by user, and SUM(amount) calculates total spending per user.

Question 43. Which query returns all users including users with no orders?

A. INNER JOIN

B. LEFT JOIN

C. RIGHT JOIN only

D. CROSS JOIN

Answer: B

Explanation:

LEFT JOIN returns all records from the left table and matching records from the right table.

Question 44. Which clause filters grouped results?

A. WHERE

B. HAVING

C. ORDER BY

D. SELECT

Answer: B

Explanation:

HAVING is used to filter aggregated/grouped results.

Question 45. Which clause filters rows before grouping?

A. WHERE

B. HAVING

C. GROUP BY

D. ORDER BY

Answer: A

Explanation:

WHERE filters rows before aggregation.

Question 46. Which key uniquely identifies a row in a table?

A. Foreign key

B. Primary key

C. Candidate value

D. Join key only

Answer: B

Explanation:

A primary key uniquely identifies each row in a table.

Question 47. What is a foreign key?

A. Key used to link two tables

B. Key used only for sorting

C. Key used to delete database

D. Key used to create frontend

Answer: A

Explanation:

A foreign key refers to the primary key of another table and maintains relationships between tables.

Question 48. Which query selects users from Bengaluru whose name is not null?

A. SELECT * FROM Users WHERE city = 'Bengaluru' AND name != NULL;

B. SELECT * FROM Users WHERE city = 'Bengaluru' AND name NOT NULL;

C. SELECT * FROM Users WHERE city = 'Bengaluru' AND name IS NOT NULL;

D. SELECT * FROM Users WHERE city = 'Bengaluru' AND name ISN'T NULL;

Answer: C

Explanation:

In SQL, null values are checked using IS NULL or IS NOT NULL.

Question 49. Difference between DELETE and TRUNCATE?

A. DELETE removes selected rows, TRUNCATE removes all rows quickly

B. DELETE drops table structure

C. TRUNCATE removes selected rows only

D. Both are exactly same

Answer: A

Explanation:

DELETE can remove specific rows using WHERE. TRUNCATE removes all rows quickly and usually resets identity counters.

Question 50. Which command removes the entire table structure?

A. DELETE

B. TRUNCATE

C. DROP

D. SELECT

Answer: C

Explanation:

DROP removes the table structure itself.

5.7 OS and Networking – Most Expected MCQs

Question 51. What is the main difference between process and thread?

A. Process is heavier, thread is lightweight unit inside process

B. Thread has separate application always

C. Both are exactly same

D. Process is only for frontend

Answer: A

Explanation:

A process has its own memory space. A thread is a lightweight execution unit inside a process and shares memory with other threads of the same process.

Question 52. What is a cookie in web applications?

A. Java object in heap

B. Small data stored by browser

C. Database table

D. Network cable

Answer: B

Explanation:

Cookies store small data in the browser, often used for session tracking, preferences, or authentication support.

Question 53. Why is in-memory cache fast?

A. It stores data in RAM

B. It stores data on tape

C. It uses HTML

D. It runs only at night

Answer: A

Explanation:

In-memory cache stores data in RAM, avoiding repeated database or disk access.

Question 54. HTTP works mainly at which OSI layer?

A. Physical

B. Network

C. Transport

D. Application

Answer: D

Explanation:

HTTP is an application layer protocol.

Question 55. TCP provides:

A. Reliable delivery

B. Image editing

C. SQL joins

D. Java compilation

Answer: A

Explanation:

TCP provides reliable, ordered delivery of data.

5.8 Git – Most Expected MCQs

Question 56. Which command shows modified and untracked files?

A. git clone

B. git diff

C. git status

D. git push

Answer: C

Explanation:

git status shows the current state of the working directory.

Question 57. Which command brings remote changes into local branch?

A. git commit

B. git pull

C. git push

D. git init

Answer: B

Explanation:

git pull fetches and merges remote changes into the local branch.

Question 58. Which command uploads local commits to remote repository?

A. git pull

B. git push

C. git clone

D. git branch

Answer: B

Explanation:

git push sends committed changes to the remote repository.

Question 59. What is a branch in Git?

A. Separate line of development

B. Database table

C. Java object

D. REST endpoint

Answer: A

Explanation:

A branch allows developers to work on changes separately without affecting the main code.

Question 60. What is a merge conflict?

A. Git cannot automatically combine changes

B. Java compiler error

C. SQL join problem

D. HTTP timeout

Answer: A

Explanation:

Merge conflict happens when different changes are made to the same part of a file.

6. Java Output-Based Questions

Question 61. What is the output?

```
int i = 0;
i = i++ + ++i;
System.out.println(i);
```

A. 1

B. 2

C. 3

D. 4

Answer: C

Explanation:

i++ uses current value first, then increments. ++i increments first, then uses value. Final result becomes 3.

Question 62. What is the output?

```
try {
    System.out.println("A");
    int x = 10 / 0;
    System.out.println("B");
} catch (ArithmeticException e) {
    System.out.println("C");
} finally {
    System.out.println("D");
}
```

A. A B C D

B. A C D

C. A D

D. C D

Answer: B

Explanation:

A prints first. Division by zero causes ArithmeticException. Control goes to catch and prints C. Finally block prints D.

Question 63. What is the output?

```
class Test {
    static {
        System.out.println("Static");
    }

    public static void main(String[] args) {
        System.out.println("Main");
    }
```

}

A. Main

B. Static

C. Static Main

D. Main Static

Answer: C

Explanation:

Static block executes before main method when the class is loaded.

Question 64. What is the output?

```
String s = "Java";
s.concat("Backend");
System.out.println(s);
```

A. JavaBackend

B. Java

C. Backend

D. Error

Answer: B

Explanation:

String is immutable. concat() creates a new String, but it is not assigned back to s.

Question 65. What is the output?

```
String s = "Java";
s = s.concat("Backend");
System.out.println(s);
```

A. JavaBackend

B. Java

C. Backend

D. Error

Answer: A

Explanation:

Here the new String is assigned back to s.

Question 66. What is the output?

```
int[] arr = {1, 2, 3};
System.out.println(arr.length);
```

A. 2

B. 3

C. length()

D. Error

Answer: B

Explanation:

Arrays use length property, not length() method.

Question 67. What is the output?

```
ArrayList<Integer> list = new ArrayList<>();
list.add(10);
list.add(20);
System.out.println(list.size());
```

A. 1

B. 2

C. 10

D. Error

Answer: B

Explanation:

Two elements are added, so size is 2.

Question 68. What is the output?

```
HashSet<Integer> set = new HashSet<>();
set.add(10);
set.add(10);
set.add(20);
System.out.println(set.size());
```

A. 1

B. 2

C. 3

D. Error

Answer: B

Explanation:

HashSet does not allow duplicates. Only 10 and 20 are stored.

7. Preparation for Technical Round

A strong technical interview answer should follow this structure:

Definition

Purpose

Small example

Practical usage

Conclusion

Do not answer only in one line.

For example, if asked “What is Spring Boot?”, do not simply say:

“Spring Boot is a framework.”

A better answer is:

“Spring Boot is a framework built on top of Spring that helps create production-ready applications with less configuration. It provides auto-configuration, embedded server support, and faster project setup. In backend development, we commonly use it to build REST APIs quickly.”

That answer feels complete.

8. Technical Interview Questions with Detailed Explanation

8.1 Core Java and OOP

Question 1. What is OOP and why is it useful in backend development?

Answer:

OOP stands for Object-Oriented Programming. It organizes code using classes and objects.

Explanation:

In backend applications, we model real-world entities such as User, Order, Product, Payment, or Request as classes. Each class contains data and behavior related to that entity.

OOP improves code readability, reusability, and maintainability.

How to explain in interview:

"OOP helps me structure backend code using classes and objects. For example, in an e-commerce backend, User, Order, and Payment can be represented as classes. This makes the code easier to understand, extend, and maintain."

What interviewer checks:

They want to know whether you understand OOP practically, not just as four pillars.

Mistake to avoid:

Do not only list inheritance, polymorphism, abstraction, and encapsulation without explanation.

Question 2. Explain class and object.

Answer:

A class is a blueprint. An object is an instance of a class.

Explanation:

A class defines variables and methods. An object is created from the class and uses those variables and methods.

Example:

```
class User {
   String name;
}

User user1 = new User();
```

Here, User is the class and user1 is the object.

How to explain in interview:

"Class defines the structure, and object is the real instance created from that structure."

Question 3. Explain inheritance.

Answer:

Inheritance allows one class to acquire properties and methods from another class.

Explanation:

It helps in code reuse. A child class can reuse common logic from the parent class and add its own behavior.

Example:

```
class Employee {
   void work() {
      System.out.println("Working");
   }
}
```

```
class Developer extends Employee {
}
```

How to explain in interview:

“Inheritance is useful when multiple classes share common behavior. We can keep common logic in the parent class and reuse it.”

Question 4. Explain polymorphism.

Answer:

Polymorphism means one name with many forms.

Explanation:

In Java, polymorphism happens through method overloading and method overriding.

Method overloading happens in the same class with different parameters.

Method overriding happens when a child class provides its own implementation of a parent class method.

How to explain in interview:

“Polymorphism allows the same method name to behave differently based on input or object type.”

Question 5. Explain abstraction.

Answer:

Abstraction means hiding implementation details and showing only necessary functionality.

Explanation:

In Java, abstraction is achieved using abstract classes and interfaces.

How to explain in interview:

“Abstraction helps expose what an object does without showing how it does it internally.”

Question 6. Explain encapsulation.

Answer:

Encapsulation means binding data and methods together and restricting direct access to data.

Explanation:

It is usually achieved using private variables and public getters/setters.

How to explain in interview:

“Encapsulation protects data and allows controlled access through methods.”

8.2 Collections

Question 7. Explain ArrayList and LinkedList.

Answer:

ArrayList is backed by a dynamic array. LinkedList is backed by nodes.

Explanation:

ArrayList is better for accessing elements by index. LinkedList is better when frequent insertion and deletion are needed.

How to explain in interview:

“In backend APIs, I usually prefer ArrayList when I need to store and iterate data. LinkedList is useful when insertions and deletions are frequent.”

Question 8. Explain HashMap.

Answer:

HashMap stores data in key-value pairs.

Explanation:

It is useful when we want to quickly retrieve a value using a key.

Example:

```
Map<Integer, String> users = new HashMap<>();
users.put(1, "Ravi");
```

How to explain in interview:

“HashMap is useful when I need fast lookup based on a key, such as finding user details by user ID.”

Question 9. Difference between HashMap and HashSet.

Answer:

HashMap stores key-value pairs. HashSet stores only unique values.

Explanation:

HashSet internally uses hashing to avoid duplicates. HashMap is used when data has a key and value relationship.

How to explain in interview:

“I use HashSet when uniqueness is needed and HashMap when I need to map one value to another.”

8.3 Exception Handling

Question 10. What is exception handling in Java?

Answer:

Exception handling is used to manage runtime errors so that the application does not crash unexpectedly.

Explanation:

Java provides try, catch, finally, throw, and throws for exception handling.

In backend services, exceptions may happen due to invalid input, database failure, network timeout, or missing data.

How to explain in interview:

“In backend systems, exceptions are normal. I use exception handling to log the error, return a proper HTTP response, and keep the application stable.”

Question 11. Difference between checked and unchecked exceptions.

Answer:

Checked exceptions are checked at compile time. Unchecked exceptions occur at runtime.

Explanation:

Examples of checked exceptions are IOException and SQLException. Examples of unchecked exceptions are NullPointerException and ArithmeticException.

How to explain in interview:

“Checked exceptions must be handled or declared, while unchecked exceptions usually occur due to programming mistakes.”

Question 12. What is finally block?

Answer:

The finally block is used to execute cleanup code regardless of whether an exception occurs or not.

Explanation:

It is commonly used to close files, database connections, or other resources.

How to explain in interview:

“finally is useful for cleanup operations that must happen even if an error occurs.”

8.4 Spring Boot and REST

Question 13. What is Spring Boot?

Answer:

Spring Boot is a framework built on top of Spring that simplifies the development of production-ready Java applications.

Explanation:

It reduces configuration effort using auto-configuration, embedded servers, and starter dependencies.

How to explain in interview:

“Spring Boot helps developers build REST APIs quickly with less setup and configuration.”

Question 14. What is REST API?

Answer:

REST API is a way for applications to communicate over HTTP using methods like GET, POST, PUT, and DELETE.

Explanation:

REST APIs expose backend functionality through endpoints.

Example:

GET /users fetches users.

POST /users creates a user.

How to explain in interview:

“REST APIs allow frontend or other systems to communicate with backend services using HTTP.”

Question 15. Explain controller, service, and repository layers.

Answer:

Controller handles HTTP requests.

Service contains business logic.

Repository interacts with the database.

Explanation:

This layered structure keeps code clean and maintainable.

How to explain in interview:

“I prefer keeping controller thin, service focused on logic, and repository responsible for database operations.”

Question 16. What is @RestController?

Answer:

@RestController is used to create REST APIs that return data directly, usually in JSON format.

Explanation:

It combines @Controller and @ResponseBody.

How to explain in interview:

“I use @RestController when I want my controller methods to return JSON responses.”

Question 17. What is @RequestBody?

Answer:

@RequestBody maps JSON request body to a Java object.

Example:

```
@PostMapping("/users")
public User createUser(@RequestBody User user) {
   return user;
}
```

How to explain in interview:

“When frontend sends JSON data, @RequestBody helps convert it into a Java object.”

Question 18. What is @PathVariable?

Answer:

@PathVariable extracts values from the URL path.

Example:

```
@GetMapping("/users/{id}")
public User getUser(@PathVariable int id) {
   return userService.getUser(id);
}
```

How to explain in interview:

"I use @PathVariable when a value is part of the URL path."

Question 19. What is @RequestParam?

Answer:

@RequestParam extracts query parameters from the URL.

Example:

/users?city=Bengaluru

How to explain in interview:

"I use @RequestParam when the value comes as a query parameter."

Question 20. What is JPA/Hibernate?

Answer:

JPA is a specification for object-relational mapping. Hibernate is an implementation of JPA.

Explanation:

They help map Java objects to database tables.

How to explain in interview:

"JPA/Hibernate help reduce manual SQL writing by mapping Java entities to database tables."

8.5 SQL and Database

Question 21. What is a primary key?

Answer:

A primary key uniquely identifies each row in a table.

Explanation:

It cannot be duplicate and usually cannot be null.

How to explain in interview:

"Primary key helps uniquely identify records, such as user_id in a Users table."

Question 22. What is a foreign key?

Answer:

A foreign key is used to link two tables.

Explanation:

It refers to the primary key of another table.

Example:

Orders.user_id can refer to Users.id.

How to explain in interview:

"Foreign key helps maintain relationship between tables."

Question 23. Difference between INNER JOIN and LEFT JOIN.

Answer:

INNER JOIN returns matching records from both tables. LEFT JOIN returns all records from the left table and matching records from the right table.

How to explain in interview:

“If I want all users even if they have no orders, I use LEFT JOIN.”

Question 24. What is indexing?

Answer:

Indexing is a database technique used to improve search performance.

Explanation:

Indexes help the database find rows faster, especially on columns used in WHERE, JOIN, or ORDER BY.

How to explain in interview:

“Indexing improves read performance, but too many indexes can slow down write operations.”

8.6 OS, Threads, and Networking

Question 25. Difference between process and thread.

Answer:

A process is an independent program in execution. A thread is a lightweight unit of execution inside a process.

Explanation:

Threads share memory within the same process, while processes have separate memory.

How to explain in interview:

“Thread is lighter than process and is useful for parallel execution inside an application.”

Question 26. What is multithreading?

Answer:

Multithreading allows multiple threads to run within the same program.

Explanation:

It helps perform multiple tasks concurrently, such as processing requests or background jobs.

How to explain in interview:

“Multithreading improves performance when tasks can run independently or concurrently.”

Question 27. What is HTTP?

Answer:

HTTP is a protocol used for communication between client and server.

Explanation:

Web applications use HTTP to send requests and receive responses.

How to explain in interview:

“HTTP is the foundation of REST API communication.”

Question 28. What are common HTTP status codes?

Answer:

200 means success.

201 means created.

400 means bad request.

401 means unauthorized.

403 means forbidden.

404 means not found.

500 means internal server error.

How to explain in interview:

“Status codes help the client understand the result of the API request.”

9. Project Discussion Preparation

In fresher backend interviews, project discussion is very important.

Even if your project is academic or training-based, explain it properly.

Use this structure:

Project name

Problem it solves

Technologies used

Modules/features

Database design

APIs created

Your role

One challenge

One improvement

Sample Project Explanation Format

“My project is a Bank Management System built using Java, Spring Boot, and MySQL. It allows users to register, login, view account details, transfer amount, and check balance. I used controller, service, and repository layers. The controller handled API requests, the service layer contained business logic, and the repository layer interacted with the database. One challenge I faced was validating transfer amount and handling insufficient balance. I solved it by adding proper checks in the service layer and returning meaningful responses.”

This sounds much stronger than simply saying:

“I did bank project using Spring Boot.”

10. Scenario-Based Technical Questions

Scenario questions test how you think.

Use this pattern:

Understand the issue

Check logs

Check recent changes

Check input

Check configuration

Check database

Check network

Fix and verify

Scenario 1. API returns 500 Internal Server Error. What will you do?

Answer:

I will first identify which API endpoint is failing and what input caused the failure. Then I will check application logs and stack trace. I will identify whether the issue happened in controller, service, repository, or database layer. I will also check database connectivity, null values, invalid input, and recent code changes.

Explanation:

500 usually means server-side error. It may happen due to unhandled exception, database issue, null pointer, invalid logic, or external dependency failure.

Strong Interview Line:

"I will not guess randomly. I will check logs and stack trace to identify the exact failure point."

What interviewer checks:

They want to see whether you debug logically.

Mistake to avoid:

Do not say only "I will restart the server."

Scenario 2. Application works locally but fails on test server.

Answer:

I will compare environment variables, application properties, database URL, Java version, dependency versions, and server logs. I will also check whether required services like database or external APIs are reachable from the test server.

Explanation:

Local and test environments may differ in configuration, database access, ports, or dependencies.

Strong Interview Line:

“I will compare local and test environment configurations to identify the difference.”

Scenario 3. Database query is slow for one API.

Answer:

I will check the SQL query being executed, whether indexes exist on filter columns, whether too much data is fetched, whether pagination is used, and whether joins are optimized.

Explanation:

Slow APIs often happen because of inefficient queries, missing indexes, large data fetch, or unnecessary joins.

Strong Interview Line:

“I will first identify the exact query and then check indexing, filters, and pagination.”

Scenario 4. API works sometimes and fails sometimes.

Answer:

I will check logs, error frequency, server resource usage, database connection pool, thread pool, network timeouts, and external service availability.

Explanation:

Intermittent failures may happen due to load, timeout, race conditions, connection limits, or external dependency issues.

Strong Interview Line:

“I will look for patterns in logs and metrics instead of treating it as a random issue.”

Scenario 5. User sends invalid JSON request.

Answer:

I will validate the request body and return a proper 400 Bad Request response with a meaningful error message.

Explanation:

Invalid client input should not become a 500 server error.

Strong Interview Line:

“Client-side input errors should be handled with proper validation and meaningful responses.”

Scenario 6. NullPointerException occurs in service layer.

Answer:

I will check which object is null from the stack trace. Then I will verify whether the object was properly initialized, whether database returned null, or whether input validation was missing.

Explanation:

NullPointerException usually happens when code tries to access a method or field on a null reference.

Strong Interview Line:

“I will identify the null object from the stack trace and fix the root cause instead of adding random null checks everywhere.”

Scenario 7. Multiple users report slow response during peak traffic.

Answer:

I will check CPU, memory, thread usage, database load, slow queries, API response time, and connection pool usage. I will also check whether caching or pagination can help.

Explanation:

Peak traffic can expose performance issues in backend systems.

Strong Interview Line:

"I will check both application-level and database-level performance."

Scenario 8. Git merge conflict occurs.

Answer:

I will open the conflicted file, compare both changes, keep the correct version, remove conflict markers, test the code, add the resolved file, and commit the fix.

Explanation:

Merge conflicts are normal in team development.

Strong Interview Line:

"I will resolve the conflict carefully and test before committing."

11. Last-Minute Most Important Questions

Before the written test and interview, revise these first:

OOP pillars with examples

Class vs object

this vs super

Method overloading vs overriding

String immutability

String equals vs ==

ArrayList vs LinkedList

HashMap vs HashSet

Checked vs unchecked exceptions

try-catch-finally behavior

throw vs throws

Spring Boot purpose

@RestController

@RequestBody

@PathVariable

@RequestParam

Controller-service-repository layers

GET vs POST vs PUT vs DELETE

HTTP status codes

Primary key vs foreign key

INNER JOIN vs LEFT JOIN

DELETE vs TRUNCATE vs DROP

Process vs thread

Cookie and cache basics

Git status, pull, push, branch, merge conflict

12. Final Preparation Strategy

For Written Test

Revise MCQs topic-wise.

Start with Core Java and OOP.

Then move to Collections and Exceptions.

Then revise Spring Boot and REST.

Then practice SQL joins and grouping.

Then revise OS, networking, and Git.

Finally, practice Java output questions.

Do not memorize blindly.

Understand why the answer is correct.

For Technical Interview

Practice speaking answers aloud.

Use this structure:

Definition

Purpose

Example

Backend usage

Conclusion

For scenario questions, use this structure:

Understand

Check logs

Check input

Check configuration

Check database

Check recent changes

Fix and verify

13. Final Encouragement

This internship does not require you to be perfect.

It requires you to be clear.

The interviewer will mainly check whether you understand Java backend fundamentals and whether you can think logically.

You do not need to know every Spring annotation.

You do not need to know advanced microservices.

You do not need to know production-level performance tuning.

But you should clearly explain:

How Java OOP works

How collections are used

How exceptions are handled

How REST APIs work

How Spring Boot layers are structured

How SQL tables are related

How HTTP status codes are used

How to debug backend failures

Your final mindset should be:

“I know the fundamentals. I can explain clearly. I can debug step by step. I am ready to learn backend systems practically.”

That is exactly what this role needs.

14. Java Coding Questions for Practice

These coding questions are enough for the Java backend internship level.

They cover Core Java, OOP thinking, Collections, Exception Handling, SQL-related logic, REST-style data processing, debugging mindset, and problem-solving.

The goal is not to write very complex algorithms.

The goal is to write:

Clean code

Readable logic

Correct output

Good variable names

Proper edge-case handling

Question 1. Count Frequency of Each Character in a String

Problem

Write a Java program to count the frequency of each character in a string.

Code

```
import java.util.*;

public class CharacterFrequency {
    public static void main(String[] args) {
        String input = "backend";

        Map<Character, Integer> frequencyMap = new LinkedHashMap<>();

        for (char ch : input.toCharArray()) {
            frequencyMap.put(ch, frequencyMap.getOrDefault(ch, 0) + 1);
        }

        for (Map.Entry<Character, Integer> entry : frequencyMap.entrySet()) {
            System.out.println(entry.getKey() + " = " + entry.getValue());
        }
    }
}
```

Explanation

This program uses a Map to store each character as a key and its count as the value.

getOrDefault() checks whether the character already exists in the map. If it exists, the count is increased. If not, the count starts from 0 and then becomes 1.

LinkedHashMap is used so that the insertion order is maintained.

Expected Output

```
b = 1
a = 1
c = 1
k = 1
e = 1
n = 1
```

d = 1

Interview Tip

This question checks your understanding of Strings, loops, Maps, and frequency logic.

Question 2. Find Duplicate Elements in an Array

Problem

Write a Java program to find duplicate elements in an integer array.

Code

```
import java.util.*;

public class DuplicateElements {
    public static void main(String[] args) {
        int[] arr = {10, 20, 30, 10, 40, 20, 50};

        Set<Integer> seen = new HashSet<>();
        Set<Integer> duplicates = new HashSet<>();

        for (int num : arr) {
            if (!seen.add(num)) {
                duplicates.add(num);
            }
        }

        System.out.println("Duplicate elements: " + duplicates);
    }
}
```

Explanation

HashSet stores only unique values.

When seen.add(num) returns false, it means the number already exists in the set. So that number is added to the duplicate set.

Expected Output

Duplicate elements: [20, 10]

Interview Tip

This question checks whether you know how to use Set for uniqueness.

Question 3. Remove Duplicates from a List

Problem

Write a Java program to remove duplicates from a list while maintaining insertion order.

Code

```
import java.util.*;

public class RemoveDuplicates {
    public static void main(String[] args) {
        List<Integer> numbers = Arrays.asList(10, 20, 10, 30, 20, 40);

        Set<Integer> uniqueNumbers = new LinkedHashSet<>(numbers);

        List<Integer> result = new ArrayList<>(uniqueNumbers);

        System.out.println(result);
    }
```

```
}
```

Explanation

LinkedHashSet removes duplicates and maintains insertion order.

First, the list is converted into a LinkedHashSet. Then it is converted back into a list.

Expected Output

[10, 20, 30, 40]

Interview Tip

This is a common Collections question.

Remember: HashSet removes duplicates but does not guarantee order. LinkedHashSet removes duplicates and maintains order.

Question 4. Find First Non-Repeating Character

Problem

Write a Java program to find the first non-repeating character in a string.

Code

```
import java.util.*;

public class FirstNonRepeatingCharacter {
    public static void main(String[] args) {
        String input = "programming";

        Map<Character, Integer> map = new LinkedHashMap<>();

        for (char ch : input.toCharArray()) {
```

```
            map.put(ch, map.getOrDefault(ch, 0) + 1);
        }

        for (Map.Entry<Character, Integer> entry : map.entrySet()) {
            if (entry.getValue() == 1) {
                System.out.println("First non-repeating character: " + entry.getKey());
                return;
            }
        }

        System.out.println("No non-repeating character found");
    }
}
```

Explanation

The first loop counts the frequency of each character.

The second loop checks which character has frequency 1.

LinkedHashMap is used because it maintains insertion order. This helps identify the first non-repeating character correctly.

Expected Output

First non-repeating character: p

Interview Tip

This question checks your ability to combine String logic with Map usage.

Question 5. Reverse Words in a Sentence

Problem

Write a Java program to reverse the words in a sentence.

Code

```
public class ReverseWords {
    public static void main(String[] args) {
        String sentence = "Java backend internship";

        String[] words = sentence.split(" ");

        StringBuilder result = new StringBuilder();

        for (int i = words.length - 1; i >= 0; i--) {
            result.append(words[i]);

            if (i != 0) {
                result.append(" ");
            }
        }

        System.out.println(result);
    }
}
```

Explanation

The sentence is split into words using split(" ").

Then the words are read from the last index to the first index and added into a StringBuilder.

StringBuilder is used because it is efficient for repeated string modifications.

Expected Output

internship backend Java

Interview Tip

This question checks your understanding of String methods, arrays, loops, and StringBuilder.

Question 6. Check Whether a String Is Palindrome

Problem

Write a Java program to check whether a string is palindrome.

Code

```
public class PalindromeCheck {
    public static void main(String[] args) {
        String input = "madam";

        int left = 0;
        int right = input.length() - 1;

        boolean isPalindrome = true;

        while (left < right) {
            if (input.charAt(left) != input.charAt(right)) {
                isPalindrome = false;
                break;
            }

            left++;
            right--;
        }

        if (isPalindrome) {
```

```
            System.out.println("Palindrome");
        } else {
            System.out.println("Not Palindrome");
        }
    }
}
```

Explanation

The program compares characters from both ends.

If all matching characters are equal, the string is a palindrome.

Expected Output

Palindrome

Interview Tip

This question checks basic logic and two-pointer thinking.

Question 7. Find Second Largest Number in an Array

Problem

Write a Java program to find the second largest number in an array.

Code

```
public class SecondLargest {
    public static void main(String[] args) {
        int[] arr = {10, 50, 20, 40, 30};

        int largest = Integer.MIN_VALUE;
        int secondLargest = Integer.MIN_VALUE;
```

```
        for (int num : arr) {
            if (num > largest) {
                secondLargest = largest;
                largest = num;
            } else if (num > secondLargest && num != largest) {
                secondLargest = num;
            }
        }

        if (secondLargest == Integer.MIN_VALUE) {
            System.out.println("Second largest number not found");
        } else {
            System.out.println("Second largest number: " + secondLargest);
        }
    }
}
```

Explanation

The program keeps track of two values:

largest

secondLargest

Whenever a new largest number is found, the old largest becomes second largest.

Expected Output

Second largest number: 40

Interview Tip

Avoid sorting if the interviewer asks for an optimized solution.

This solution works in one pass.

Question 8. Count Words in a Sentence

Problem

Write a Java program to count the number of words in a sentence.

Code

```
public class WordCount {
    public static void main(String[] args) {
        String sentence = "Java backend development is powerful";

        String[] words = sentence.trim().split("\\s+");

        System.out.println("Word count: " + words.length);
    }
}
```

Explanation

trim() removes extra spaces from the beginning and end.

split("\\s+") splits the sentence based on one or more spaces.

This handles multiple spaces properly.

Expected Output

Word count: 5

Interview Tip

This question checks whether you can handle simple edge cases like extra spaces.

Question 9. Sort Employees by Salary Using Comparator

Problem

Write a Java program to sort employees based on salary.

Code

```
import java.util.*;

class Employee {
    int id;
    String name;
    double salary;

    Employee(int id, String name, double salary) {
        this.id = id;
        this.name = name;
        this.salary = salary;
    }

    public String toString() {
        return id + " " + name + " " + salary;
    }
}

public class SortEmployees {
    public static void main(String[] args) {
        List<Employee> employees = new ArrayList<>();

        employees.add(new Employee(1, "Ravi", 30000));
        employees.add(new Employee(2, "Anu", 45000));
        employees.add(new Employee(3, "Kiran", 25000));

        employees.sort(Comparator.comparingDouble(e -> e.salary));

        for (Employee emp : employees) {
            System.out.println(emp);
```

```
        }
    }
}
```

Explanation

This program uses a custom class Employee.

The employees are stored in an ArrayList.

Comparator.comparingDouble() is used to sort employees based on salary.

Expected Output

3 Kiran 25000.0
1 Ravi 30000.0
2 Anu 45000.0

Interview Tip

This question checks OOP, Collections, Comparator, and object sorting.

Question 10. Handle Division Using Exception Handling

Problem

Write a Java program to handle division by zero using exception handling.

Code

```
public class DivisionException {
    public static void main(String[] args) {
        int a = 10;
        int b = 0;

        try {
```

```
            int result = a / b;
            System.out.println("Result: " + result);
        } catch (ArithmeticException e) {
            System.out.println("Cannot divide by zero");
        } finally {
            System.out.println("Program execution completed");
        }
    }
}
```

Explanation

Division by zero causes ArithmeticException.

The catch block handles the exception.

The finally block executes whether an exception occurs or not.

Expected Output

Cannot divide by zero
Program execution completed

Interview Tip

This question checks your understanding of try, catch, and finally.

Question 11. Create Custom Exception for Invalid Age

Problem

Write a Java program to create a custom exception if age is less than 18.

Code

```
class InvalidAgeException extends Exception {
    InvalidAgeException(String message) {
        super(message);
    }
}

public class CustomExceptionExample {
    public static void validateAge(int age) throws InvalidAgeException {
        if (age < 18) {
            throw new InvalidAgeException("Age must be 18 or above");
        } else {
            System.out.println("Valid age");
        }
    }

    public static void main(String[] args) {
        try {
            validateAge(16);
        } catch (InvalidAgeException e) {
            System.out.println("Exception: " + e.getMessage());
        }
    }
}
```

Explanation

A custom exception is created by extending the Exception class.

The method validateAge() throws the custom exception when age is less than 18.

Expected Output

Exception: Age must be 18 or above

Interview Tip

Custom exception questions are common when the JD mentions exception handling.

Question 12. Find Users from a Particular City Using List

Problem

Write a Java program to filter users based on city.

Code

```
import java.util.*;

class User {
    int id;
    String name;
    String city;

    User(int id, String name, String city) {
        this.id = id;
        this.name = name;
        this.city = city;
    }

    public String toString() {
        return id + " " + name + " " + city;
    }
}

public class FilterUsers {
    public static void main(String[] args) {
        List<User> users = new ArrayList<>();

        users.add(new User(1, "Ravi", "Bengaluru"));
        users.add(new User(2, "Anu", "Chennai"));
```

```
        users.add(new User(3, "Kiran", "Bengaluru"));

        for (User user : users) {
            if (user.city.equalsIgnoreCase("Bengaluru")) {
                System.out.println(user);
            }
        }
    }
}
```

Explanation

This program simulates a simple backend filtering operation.

In real backend applications, this kind of filtering may happen through SQL queries, repository methods, or service-layer logic.

Expected Output

1 Ravi Bengaluru
3 Kiran Bengaluru

Interview Tip

This question connects Java Collections with backend-style data filtering.

Question 13. Convert Map Data into JSON-Like Output

Problem

Write a Java program to print user data in JSON-like format.

Code

```
import java.util.*;

public class JsonLikeOutput {
    public static void main(String[] args) {
        Map<String, Object> user = new LinkedHashMap<>();

        user.put("id", 101);
        user.put("name", "Ravi");
        user.put("city", "Bengaluru");

        StringBuilder json = new StringBuilder();

        json.append("{");

        int count = 0;

        for (Map.Entry<String, Object> entry : user.entrySet()) {
            json.append("\"").append(entry.getKey()).append("\": ");

            if (entry.getValue() instanceof String) {
                json.append("\"").append(entry.getValue()).append("\"");
            } else {
                json.append(entry.getValue());
            }

            count++;

            if (count < user.size()) {
                json.append(", ");
            }
        }

        json.append("}");

        System.out.println(json);
```

```
    }
}
```

Explanation

This question connects Java Map with JSON-style data representation.

REST APIs commonly send and receive data in JSON format.

Expected Output

{"id": 101, "name": "Ravi", "city": "Bengaluru"}

Interview Tip

This question is useful because the JD mentions REST APIs and JSON.

Question 14. Find Maximum Occurring Word in a Sentence

Problem

Write a Java program to find the word with maximum frequency.

Code

```
import java.util.*;

public class MaxOccurringWord {
    public static void main(String[] args) {
        String sentence = "java is powerful and java is popular";

        String[] words = sentence.toLowerCase().split("\\s+");

        Map<String, Integer> map = new HashMap<>();

        for (String word : words) {
```

```
            map.put(word, map.getOrDefault(word, 0) + 1);
        }

        String maxWord = "";
        int maxCount = 0;

        for (Map.Entry<String, Integer> entry : map.entrySet()) {
            if (entry.getValue() > maxCount) {
                maxWord = entry.getKey();
                maxCount = entry.getValue();
            }
        }

        System.out.println(maxWord + " = " + maxCount);
    }
}
```

Explanation

The sentence is split into words.

A HashMap stores each word and its count.

Then the map is traversed to find the word with the highest count.

Expected Output

java = 2

Interview Tip

This question checks Strings, Maps, loops, and frequency logic.

Question 15. Simple Service Layer Logic: Transfer Amount

Problem

Write a Java program to simulate amount transfer with balance validation.

Code

```
class Account {
    private int accountNumber;
    private double balance;

    Account(int accountNumber, double balance) {
        this.accountNumber = accountNumber;
        this.balance = balance;
    }

    public double getBalance() {
        return balance;
    }

    public void debit(double amount) {
        balance = balance - amount;
    }

    public void credit(double amount) {
        balance = balance + amount;
    }
}

public class TransferAmount {
    public static void transfer(Account from, Account to, double amount) {
        if (amount <= 0) {
            System.out.println("Amount must be greater than zero");
            return;
        }
```

```
        if (from.getBalance() < amount) {
            System.out.println("Insufficient balance");
            return;
        }

        from.debit(amount);
        to.credit(amount);

        System.out.println("Transfer successful");
    }

    public static void main(String[] args) {
        Account account1 = new Account(101, 5000);
        Account account2 = new Account(102, 2000);

        transfer(account1, account2, 1500);

        System.out.println("Account 1 Balance: " + account1.getBalance());
        System.out.println("Account 2 Balance: " + account2.getBalance());
    }
}
```

Explanation

This is a backend-style business logic question.

The program checks:

Amount should be greater than zero

Sender should have enough balance

Debit from one account

Credit to another account

This is similar to service-layer validation in Spring Boot applications.

Expected Output

Transfer successful
Account 1 Balance: 3500.0
Account 2 Balance: 3500.0

Interview Tip

This question is excellent for backend interviews because it checks OOP, encapsulation, validation, and business logic.

Final Coding Preparation Tip

For Java coding rounds, focus on these patterns:

String frequency

Duplicate removal

Array traversal

Map-based counting

Set-based uniqueness

Sorting objects

Exception handling

Basic OOP design

Simple backend-style validation logic

Do not write complicated code unnecessarily.

Write simple, readable, correct Java code.

In interview, always explain:

What data structure you used

Why you used it

What edge cases you handled

What the time complexity is if asked

That will make your coding answer stronger.